

Создание структуры проекта и инициализация модуля

Создадим базовые папки и файлы проекта, а также инициализируем проект Go:

```
mkdir pz3-http
cd pz3-http
mkdir -pv cmd/server internal/api internal/storage
go mod init example.com/pz3-http
```

```
falcion@DESKTOP-KCIB6QV:/mnt/d/.WORKSPACE/Education/T1.3$ mkdir pz3-http
falcion@DESKTOP-KCIB6QV:/mnt/d/.WORKSPACE/Education/T1.3$ cd pz3-http
falcion@DESKTOP-KCIB6QV:/mnt/d/.WORKSPACE/Education/T1.3/pz3-http$ mkdir -pv cmd/server internal/api internal/storage
mkdir: created directory 'cmd'
mkdir: created directory 'cmd/server'
mkdir: created directory 'internal'
mkdir: created directory 'internal/api'
mkdir: created directory 'internal/storage'
```

Создание самого проекта и его кода

Хранилище задач (internal/storage/memory.go):

```
package storage

import (
    "errors"
    "sync"
)

var ErrNotFound = errors.New("not found")

type Task struct {
    ID      int64  `json:"id"`
    Title   string `json:"title"`
    Done    bool   `json:"done"`
}

type MemoryStore struct {
    mu      sync.RWMutex
    auto    int64
    tasks   map[int64]*Task
}

func NewMemoryStore() *MemoryStore {
```

```

        return &MemoryStore{
            tasks: make(map[int64]*Task),
        }
    }

func (s *MemoryStore) Create(title string) *Task {
    s.mu.Lock()
    defer s.mu.Unlock()

    s.auto++
    t := &Task{ID: s.auto, Title: title, Done: false}
    s.tasks[t.ID] = t

    cp := *t
    return &cp
}

func (s *MemoryStore) Get(id int64) (*Task, error) {
    s.mu.RLock()
    defer s.mu.RUnlock()

    t, ok := s.tasks[id]
    if !ok {
        return nil, ErrNotFound
    }

    cp := *t
    return &cp, nil
}

func (s *MemoryStore) List() []*Task {
    s.mu.RLock()
    defer s.mu.RUnlock()

    out := make([]*Task, 0, len(s.tasks))
    for _, t := range s.tasks {
        cp := *t
        out = append(out, &cp)
    }
    return out
}

func (s *MemoryStore) UpdateDone(id int64, done bool) (*Task, error) {
    s.mu.Lock()
    defer s.mu.Unlock()

```

```

        t, ok := s.tasks[id]
        if !ok {
            return nil, ErrNotFound
        }

        t.Done = done
        cp := *t
        return &cp, nil
    }

    func (s *MemoryStore) Delete(id int64) error {
        s.mu.Lock()
        defer s.mu.Unlock()

        if _, ok := s.tasks[id]; !ok {
            return ErrNotFound
        }

        delete(s.tasks, id)
        return nil
    }

```

Объединённые вспомогательные функции для JSON-ответов и ошибок
(`internal/api/responses.go`):

```

package api

import (
    "encoding/json"
    "net/http"
)

type ErrorResponse struct {
    Error string `json:"error"`
}

func JSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)

    if v != nil {
        _ = json.NewEncoder(w).Encode(v)
    }
}

```

```

func BadRequest(w http.ResponseWriter, msg string) {
    JSON(w, http.StatusBadRequest, ErrorResponse{Error: msg})
}

func NotFound(w http.ResponseWriter, msg string) {
    JSON(w, http.StatusNotFound, ErrorResponse{Error: msg})
}

func UnprocessableEntity(w http.ResponseWriter, msg string) {
    JSON(w, http.StatusUnprocessableEntity, ErrorResponse{Error: msg})
}

func Internal(w http.ResponseWriter, msg string) {
    JSON(w, http.StatusInternalServerError, ErrorResponse{Error: msg})
}

func MethodNotAllowed(w http.ResponseWriter, msg string) {
    JSON(w, http.StatusMethodNotAllowed, ErrorResponse{Error: msg})
}

```

Обработчики для задач (`internal/api/handlers.go`):

```

package api

import (
    "encoding/json"
    "errors"
    "net/http"
    "strconv"
    "strings"

    "example.com/pz3-http/internal/storage"
)

type Handlers struct {
    Store *storage.MemoryStore
}

func NewHandlers(store *storage.MemoryStore) *Handlers {
    return &Handlers{Store: store}
}

type createTaskRequest struct {
    Title string `json:"title"`
}

```

```

type patchTaskRequest struct {
    Done *bool `json:"done"`
}

// GET /tasks?q=text
func (h *Handlers) ListTasks(w http.ResponseWriter, r *http.Request) {
    tasks := h.Store.List()

    q := strings.TrimSpace(r.URL.Query().Get("q"))
    if q != "" {
        filtered := make([]*storage.Task, 0)
        lowerQ := strings.ToLower(q)

        for _, t := range tasks {
            if strings.Contains(strings.ToLower(t.Title), lowerQ)
{
                filtered = append(filtered, t)
            }
        }

        tasks = filtered
    }

    JSON(w, http.StatusOK, tasks)
}

// POST /tasks
func (h *Handlers) CreateTask(w http.ResponseWriter, r *http.Request) {
    if ct := r.Header.Get("Content-Type"); ct != "" &&
!strings.Contains(ct, "application/json") {
        BadRequest(w, "Content-Type must be application/json")
        return
    }

    var req createTaskRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        BadRequest(w, "invalid JSON body")
        return
    }

    title := strings.TrimSpace(req.Title)
    if len([]rune(title)) < 3 || len([]rune(title)) > 140 {
        UnprocessableEntity(w, "title must be 3..140 characters")
        return
    }
}

```

```

        task := h.Store.Create(title)
        JSON(w, http.StatusCreated, task)
    }

// GET /tasks/{id}
func (h *Handlers) GetTask(w http.ResponseWriter, r *http.Request) {
    id, ok := parseIDFromPath(w, r)
    if !ok {
        return
    }

    task, err := h.Store.Get(id)
    if err != nil {
        if errors.Is(err, storage.ErrNotFound) {
            NotFound(w, "task not found")
            return
        }
        Internal(w, "unexpected error")
        return
    }

    JSON(w, http.StatusOK, task)
}

// PATCH /tasks/{id}
func (h *Handlers) PatchTask(w http.ResponseWriter, r *http.Request) {
    id, ok := parseIDFromPath(w, r)
    if !ok {
        return
    }

    var req patchTaskRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        BadRequest(w, "invalid JSON body")
        return
    }

    if req.Done == nil {
        BadRequest(w, "field done is required")
        return
    }

    task, err := h.Store.UpdateDone(id, *req.Done)
    if err != nil {
        if errors.Is(err, storage.ErrNotFound) {

```

```

        NotFound(w, "task not found")
        return
    }
    Internal(w, "unexpected error")
    return
}

JSON(w, http.StatusOK, task)
}

// DELETE /tasks/{id}
func (h *Handlers) DeleteTask(w http.ResponseWriter, r *http.Request) {
    id, ok := parseIDFromPath(w, r)
    if !ok {
        return
    }

    if err := h.Store.Delete(id); err != nil {
        if errors.Is(err, storage.ErrNotFound) {
            NotFound(w, "task not found")
            return
        }
        Internal(w, "unexpected error")
        return
    }

    w.WriteHeader(http.StatusNoContent)
}

func parseIDFromPath(w http.ResponseWriter, r *http.Request) (int64, bool) {
    path := strings.Trim(r.URL.Path, "/")
    parts := strings.Split(path, "/")

    if len(parts) != 2 || parts[0] != "tasks" {
        NotFound(w, "invalid path")
        return 0, false
    }

    id, err := strconv.ParseInt(parts[1], 10, 64)
    if err != nil {
        BadRequest(w, "invalid id")
        return 0, false
    }

    return id, true
}

```

Прослойка для логирования и CORS (internal/api/middleware.go):

```
package api

import (
    "log"
    "net/http"
    "time"
)

type statusRecorder struct {
    http.ResponseWriter
    status int
}

func (r *statusRecorder) WriteHeader(code int) {
    r.status = code
    r.ResponseWriter.WriteHeader(code)
}

func (r *statusRecorder) Write(b []byte) (int, error) {
    if r.status == 0 {
        r.status = http.StatusOK
    }
    return r.ResponseWriter.Write(b)
}

func Logging(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        rec := &statusRecorder{ResponseWriter: w, status:
http.StatusOK}

        next.ServeHTTP(rec, r)

        log.Printf("%s %s %d %s", r.Method, r.URL.Path, rec.status,
time.Since(start))
    })
}

func CORS(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        w.Header().Set("Access-Control-Allow-Origin", "*")
        w.Header().Set("Access-Control-Allow-Methods", "GET, POST,
PATCH, DELETE, OPTIONS")
        w.Header().Set("Access-Control-Allow-Headers", "Content-Type,
```

```

Authorization")

        if r.Method == http.MethodOptions {
            w.WriteHeader(http.StatusNoContent)
            return
        }

        next.ServeHTTP(w, r)
    })
}

```

И само ядро сервера:

```

package main

import (
    "context"
    "errors"
    "log"
    "net/http"
    "os"
    "os/signal"
    "syscall"
    "time"

    "example.com/pz3-http/internal/api"
    "example.com/pz3-http/internal/storage"
)

func main() {
    store := storage.NewMemoryStore()
    h := api.NewHandlers(store)

    mux := http.NewServeMux()

    mux.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request)
{
        if r.Method != http.MethodGet {
            api.MethodNotAllowed(w, "method not allowed")
            return
        }
        api.JSON(w, http.StatusOK, map[string]string{"status": "ok"})
    })

    mux.HandleFunc("/tasks", func(w http.ResponseWriter, r *http.Request)

```

```

{
    switch r.Method {
    case http.MethodGet:
        h.ListTasks(w, r)
    case http.MethodPost:
        h.CreateTask(w, r)
    default:
        api.MethodNotAllowed(w, "method not allowed")
    }
})

mux.HandleFunc("/tasks/", func(w http.ResponseWriter, r *http.Request)
{
    switch r.Method {
    case http.MethodGet:
        h.GetTask(w, r)
    case http.MethodPatch:
        h.PatchTask(w, r)
    case http.MethodDelete:
        h.DeleteTask(w, r)
    default:
        api.MethodNotAllowed(w, "method not allowed")
    }
})

handler := api.Logging(api.CORS(mux))

addr := ":8080"
if port := os.Getenv("PORT"); port != "" {
    addr = ":" + port
}

srv := &http.Server{
    Addr:    addr,
    Handler: handler,
}

go func() {
    log.Println("listening on", addr)
    if err := srv.ListenAndServe(); err != nil && !errors.Is(err,
http.ErrServerClosed) {
        log.Fatalf("server error: %v", err)
    }
}()

quit := make(chan os.Signal, 1)

```

```

    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)
    <-quit

    log.Println("shutting down...")

    ctx, cancel := context.WithTimeout(context.Background(),
5*time.Second)
    defer cancel()

    if err := srv.Shutdown(ctx); err != nil {
        log.Fatal("server forced to shutdown:", err)
    }

    log.Println("server stopped")
}

```

Запуск сервера

Запустим сервер в режиме отладки:

```
go run ./cmd/server
```

```

ge/go-get=1: 404 Not Found
PS D:\WORKSPACE\Education\T1.3\pz3-http> go mod tidy
PS D:\WORKSPACE\Education\T1.3\pz3-http> go run ./cmd/server
2026/09/25 20:04:00 listening on :8080

```

Тестирование

GET /health:

```
curl.exe -i http://localhost:8080/health
```

```

PS C:\Users\Falcion> curl.exe -i http://localhost:8080/health
HTTP/1.1 200 OK
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Methods: GET, POST, PATCH, DELETE, OPTIONS
Access-Control-Allow-Origin: *
Content-Type: application/json
Date: Fri, 25 Sep 2026 17:05:08 GMT
Content-Length: 16

{"status": "ok"}
PS C:\Users\Falcion>

```

Создание задачи:

```
Invoke-RestMethod -Method Post -Uri http://localhost:8080/tasks -ContentType "application/json" -Body '{"title":"Buy milk"}'
```

```
[Error]: Invalid JSON body }
PS C:\Users\Falcion> Invoke-RestMethod -Method Post -Uri http://localhost:8080/tasks -ContentType "application/json" -Body '{"title":"Buy milk"}'

id title    done
-- -----
1 Buy milk False
```

Список задач:

```
Invoke-RestMethod http://localhost:8080/tasks
```

```
PS C:\Users\Falcion> Invoke-RestMethod http://localhost:8080/tasks

id title    done
-- -----
1 Buy milk False
```

Фильтр (просмотр списка):

```
Invoke-RestMethod "http://localhost:8080/tasks?q=Buy"
```

```
PS C:\Users\Falcion> Invoke-RestMethod "http://localhost:8080/tasks?q=Buy"

id title    done
-- -----
1 Buy milk False
```

Вызов задачи по ID:

```
Invoke-RestMethod http://localhost:8080/tasks/1
```

```
PS C:\Users\Falcion> Invoke-RestMethod http://localhost:8080/tasks/1

id title    done
-- -----
1 Buy milk False
```

Метод PATCH (обновление данных):

```
Invoke-RestMethod -Method Patch -Uri http://localhost:8080/tasks/1 -
ContentType "application/json" -Body '{"done":true}'
```

```
PS C:\Users\Falcion> Invoke-RestMethod -Method Patch -Uri http://localhost:8080/tasks/1 -ContentType "application/json"
-Body '{"done":true}'

id title    done
--
1 Buy milk True
```

Метод DELETE:

```
Invoke-RestMethod -Method Delete -Uri http://localhost:8080/tasks/1
```

```
PS C:\Users\Falcion> Invoke-RestMethod -Method Delete -Uri http://localhost:8080/tasks/1
PS C:\Users\Falcion>
```

ничего не возвращается, т.к. 204 код - норма для DELETE методов

P.S.: мы использовали `Invoke-RestMethod` ибо работаем в PowerShell и для вставки каких-то разумных JSON-объектов в CLI лучше использовать его, чем разбивать в многострочные команды `curl` запросы.

Проверим "кривые" запросы:

```
PS C:\Users\Falcion> curl.exe -i http://localhost:8080/tasks/abc
HTTP/1.1 400 Bad Request
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Methods: GET, POST, PATCH, DELETE, OPTIONS
Access-Control-Allow-Origin: *
Content-Type: application/json
Date: Fri, 02 Oct 2026 17:23:24 GMT
Content-Length: 23

{"error":"invalid id"}
PS C:\Users\Falcion>
```

```
PS C:\Users\Falcion> curl.exe -i http://localhost:8080/tasks/9999
HTTP/1.1 404 Not Found
Access-Control-Allow-Headers: Content-Type, Authorization
Access-Control-Allow-Methods: GET, POST, PATCH, DELETE, OPTIONS
Access-Control-Allow-Origin: *
Content-Type: application/json
Date: Fri, 02 Oct 2026 17:24:25 GMT
Content-Length: 27

{"error":"task not found"}
```

ЮНИТ-ТЕСТЫ

Создадим (`internal/api/handlers_test.go`):

```
package api

import (
    "bytes"
    "encoding/json"
    "net/http"
    "net/http/httptest"
    "testing"

    "example.com/pz3-http/internal/storage"
)

func TestCreateTask(t *testing.T) {
    store := storage.NewMemoryStore()
    h := NewHandlers(store)

    body := []byte(`{"title":"Купить молоко"}`)
    req := httptest.NewRequest(http.MethodPost, "/tasks",
bytes.NewReader(body))
    req.Header.Set("Content-Type", "application/json")
    rr := httptest.NewRecorder()

    h.CreateTask(rr, req)

    if rr.Code != http.StatusCreated {
        t.Fatalf("expected 201, got %d", rr.Code)
    }

    var task storage.Task
    if err := json.NewDecoder(rr.Body).Decode(&task); err != nil {
        t.Fatal(err)
    }

    if task.ID != 1 || task.Title != "Купить молоко" || task.Done {
        t.Fatalf("unexpected task: %+v", task)
    }
}

func TestCreateTaskValidation(t *testing.T) {
    store := storage.NewMemoryStore()
    h := NewHandlers(store)
```

```

    body := []byte(`{"title":"ab"}`)
    req := httptest.NewRequest(http.MethodPost, "/tasks",
bytes.NewReader(body))
    req.Header.Set("Content-Type", "application/json")
    rr := httptest.NewRecorder()

    h.CreateTask(rr, req)

    if rr.Code != http.StatusUnprocessableEntity {
        t.Fatalf("expected 422, got %d", rr.Code)
    }
}

func TestListTasksFilter(t *testing.T) {
    store := storage.NewMemoryStore()
    store.Create("Купить молоко")
    store.Create("Сделать зарядку")

    h := NewHandlers(store)

    req := httptest.NewRequest(http.MethodGet, "/tasks?q=молоко", nil)
    rr := httptest.NewRecorder()

    h.ListTasks(rr, req)

    if rr.Code != http.StatusOK {
        t.Fatalf("expected 200, got %d", rr.Code)
    }

    var tasks []storage.Task
    if err := json.NewDecoder(rr.Body).Decode(&tasks); err != nil {
        t.Fatal(err)
    }

    if len(tasks) != 1 || tasks[0].Title != "Купить молоко" {
        t.Fatalf("unexpected tasks: %+v", tasks)
    }
}

```

Запустим тесты:

```
go test ./...
```

```
2026/10/02 20:30:53 server stopped
PS D:\.WORKSPACE\Education\T1.3\pz3-http> go test ./...
?      example.com/pz3-http/cmd/server [no test files]
ok     example.com/pz3-http/internal/api    0.713s
?      example.com/pz3-http/internal/storage [no test files]
PS D:\.WORKSPACE\Education\T1.3\pz3-http> |
```